

Introduction to



with Application to Bioinformatics

- Day 3

Day 3

- **Session 1**
 - Quiz: Review of Day 2
 - Lecture: Go through questions, data type `set`
 - Ex1: IMDb exercise - Find the number of unique genres
- **Session 2**
 - Lecture: Data type `dict`
 - Ex2: IMDb exercise - Find the number of movies per genre
 - PyQuiz 3.1
- **Session 3**
 - Lecture: Write you own functions
 - Ex3: Day 3, Exercise 3, Functions
- **Session 4**
 - Lecture: Pass arguments from command line using `sys.argv` and string formatting
 - Ex4: IMDb exercise - functions and `sys.argv`
 - PyQuiz 3.2
- **Project time**

Quiz: Review Day 2

Go to Canvas, Modules -> Day 3 -> Review Day 2

~20 minutes

Tuples (Q 1&2)

1. Which of the following variables are of the type tuple?

```
a = (1, 2, 3, 4)
```

```
a = ([1, 2], 'a', 'b')
```

2. What is the difference between a tuple and a list?

A tuple is immutable while a list is mutable

```
In [1]: myTuple    = (1, 2, 3)
        myList     = [1, 2, 3]
```

2. What is the difference between a tuple and a list?

A tuple is immutable while a list is mutable

```
In [1]: myTuple    = (1, 2, 3)
        myList     = [1, 2, 3]
```

```
In [12]: myList[2] = 4
        myList
```

```
Out[12]: [1, 2, 4]
```

2. What is the difference between a tuple and a list?

A tuple is immutable while a list is mutable

```
In [1]: myTuple    = (1, 2, 3)
        myList     = [1, 2, 3]
```

```
In [12]: myList[2] = 4
        myList
```

```
Out[12]: [1, 2, 4]
```

```
In [13]: myTuple[2] = 4
```

```
-----
-----
TypeError
```

```
call last)
```

```
Cell In[13], line 1
```

```
----> 1 myTuple[2] = 4
```

Traceback (most recent

```
TypeError: 'tuple' object does not support item assignment
```

Is it true that we can never modify the content of a tuple?

Is it true that we can never modify the content of a tuple?

```
In [14]: myTuple = (1, 2, [1,2,3])  
print(myTuple)  
myTuple[2][2] = 4  
print(myTuple)
```

```
(1, 2, [1, 2, 3])
```

```
(1, 2, [1, 2, 4])
```

Is it true that we can never modify the content of a tuple?

```
In [14]: myTuple = (1, 2, [1,2,3])  
print(myTuple)  
myTuple[2][2] = 4  
print(myTuple)
```

```
(1, 2, [1, 2, 3])  
(1, 2, [1, 2, 4])
```

- The immutability of tuples in Python means that **the structure of the tuple itself cannot be changed**, you cannot add, remove, or replace elements in the tuple.
- However, if a tuple contains mutable objects like lists, dictionaries, or other objects, the contents of those mutable objects can still be changed.

How to structure the code (Q 3)

3. What does pseudocode mean?

Writing down the steps you intend to include in your code in more general language

Things to Consider When Writing Pseudocode

- Decide on the desired output.
- Identify the input files you have.
- Examine the structure of the input – can it be iterated over?
- Determine where the necessary information is located.
- Assess if you need to store information while iterating:
 - Use lists for ordered data.
 - Use sets for unique, non-duplicate entries.
 - Use dictionaries for structured, key-value information.
- After collecting the required data, decide how to process it.
- Determine if you'll need to write your results to a file.

Writing pseudocode before actual coding is a good habit.

Functions and methods (Q 4&5)

4. What are the following examples of?

```
len([1, 2, 3, 4])  
print("my text")
```

Functions

Functions and methods (Q 4&5)

4. What are the following examples of?

```
len([1, 2, 3, 4])  
print("my text")
```

Functions

5. What are the following examples of?

```
"my\ttext".split("\t")  
[1, 2, 3].pop()
```

Methods

What are the differences between a **function** and a **method**?

Function	Method
Standalone block of code	Function associated with an object
Called independently, e.g. <code>functionName()</code>	Called on an instance of a class, e.g. <code>obj.methodName()</code>
Not tied to any object or class	Tied to the objects they are called on
Defined outside of a class	Defined within a class

|

6. Calculate the average of the list `[1,2,3.5,5,6.2]` to one decimal, using Python

```
In [3]: myList = [1, 2, 3.5, 5, 6.2]  
        round(sum(myList)/len(myList),1)
```

```
Out[3]: 3.5
```


7. Take the list `['I', 'know', 'Python']` as input and output the string 'I KNOW PYTHON'

```
In [4]: my_list = ['I', 'know', 'Python']  
        " ".join(my_list).upper()
```

```
Out[4]: 'I KNOW PYTHON'
```

Exercise from day 2

Find the movie with the highest rating in the file
250.imdb

#	Votes	Rating	Year	Runtime	URL	Genres	Title
126807		8.5	1957	5280	https://images-na.ssl-images...	Drama,War	Paths of Glory
71379		8.2	1925	4320	https://images-na.ssl-images...	Adventure,Comedy,Drama,Family	The Gold

```
In [20]: # Code Snippet for Finding the Movie with the Highest Rating
with open('../downloads/250.imdb', 'r') as fh:
    #Initialize empty list and set rating variable to 0
    movieList = []
    highestRating = 0

    # Iterate through the lines of the file and extract
    # the relevant variables - create a tuple for each line
    for line in fh:
        if not line.startswith('#'):
            cols = line.strip().split('|')
            rating = float(cols[1].strip())
            title = cols[6].strip()
            movieList.append((rating, title))

            # If rating of current movie > rating variable,
            # reassign rating variable
            if rating > highestRating:
                highestRating = rating
    print("Movie(s) with highest rating " + str(highestRating) + ":" )

    # Extract title of movie with highest Score
    for i in range(len(movieList)):
        if movieList[i][0] == highestRating:
            print(movieList[i][1])
```

Movie(s) with highest rating 9.3:
The Shawshank Redemption

The `with` key word

Use the `with` keyword to ensure the file handle be closed automatically, instead of having to call `.close()` This helps managing compute resources and ensures data integrity.

```
file = open("filename.txt", "r")
content = file.read()
file.close()
```

```
with open("filename.txt", "r") as file:
    content = file.read()
```

```
with open ("filename.txt", "r", encoding='utf-8') as file
    content = file.read()
```

However, for Python 3, the default encoding is usually 'utf-8', so it's not needed.

New data type: **set**

- A set contains an unordered collection of unique and hashable objects
 - **Unordered:** Items have no defined order
 - **Unique:** Duplicate items are not allowed
 - **Hashable:** Each item must be hashable

Syntax:

```
setName = set() # Create an empty set
```

```
setName = {1,2,3,4,5} # Create a populated set  
setName = set([1,2,3,4,5]) # Alternative way
```

A set is unordered

```
In [1]: mySet = {1,5,2,3,4}
        for e in mySet:
            print(e)
```

```
1
2
3
4
5
```

A **set** contains only unique elements

```
In [24]: mySet = {"1", "1", "2", "2", "3"}  
         print(mySet)
```

```
{'2', '3', '1'}
```

A set can only have hashable / immutable elements

```
In [34]: mySet = {1, "tga", (3, 4), 5.6, False}  
         print(mySet)
```

```
{False, 1, 'tga', 5.6, (3, 4)}
```


A **set** can only have hashable / immutable elements

```
In [34]: mySet = {1, "tga", (3, 4), 5.6, False}  
print(mySet)
```

```
{False, 1, 'tga', 5.6, (3, 4)}
```

Trying to include a mutable element:

A **set** can only have hashable / immutable elements

```
In [34]: mySet = {1, "tga", (3, 4), 5.6, False}
          print(mySet)
```

```
{False, 1, 'tga', 5.6, (3, 4)}
```

Trying to include a mutable element:

```
In [36]: mySet = {1, "tga", [3, 4], 5.6, False}
```

```
-----
-----
TypeError                                Traceback (most recent
call last)
Cell In[36], line 1
----> 1 mySet = {1, "tga", [3, 4], 5.6, False}

TypeError: unhashable type: 'list'
```

Basic operations on **set**

In [5]: *# Add elements to a set*

```
myset = set()
myset.add(1)
myset.add(100)
myset.add(100)
myset.add(100)

print(myset)
```

```
{1, 100}
```

Basic operations on set

In [5]: *# Add elements to a set*

```
myset = set()
myset.add(1)
myset.add(100)
myset.add(100)
myset.add(100)

print(myset)
```

{1, 100}

In [5]: *# get the number of elements of a set*

```
len(myset)
```

Out[5]: 2

Basic operations on set

In [5]: *# Add elements to a set*

```
myset = set()
myset.add(1)
myset.add(100)
myset.add(100)
myset.add(100)

print(myset)
```

{1, 100}

In [5]: *# get the number of elements of a set*

```
len(myset)
```

Out[5]: 2

In [6]: *# membership checking*

```
12 in myset
```

Out[6]: False

Basic operations on set

```
In [5]: # Add elements to a set  
myset = set()  
myset.add(1)  
myset.add(100)  
myset.add(100)  
myset.add(100)  
  
print(myset)
```

{1, 100}

```
In [5]: # get the number of elements of a set  
len(myset)
```

Out[5]: 2

```
In [6]: # membership checking  
12 in myset
```

Out[6]: False

Learn more on https://www.w3schools.com/python/python_sets.asp

When the size of list is large, membership checking with `set` tends to be much faster than with `list`

```
In [8]: import time, random

# Create a large list and set
large_list = list(range(10000000))
large_set = set(large_list)
elements_to_find = random.sample(range(10000001), 10)

# Measure time for list membership check
list_time = time.time()
for e in elements_to_find:
    e in large_list
list_time = time.time() - list_time

# Measure time for set membership check
set_time = time.time()
for e in elements_to_find:
    e in large_set
set_time = time.time() - set_time

print(f"List check: {list_time:.6f} seconds")
print(f"Set check: {set_time:.6f} seconds")
print(f"Set is approximately {list_time / set_time:.2f} times faster.")
```

List check: 0.233157 seconds

Set check: 0.000033 seconds

Set is approximately 7086.46 times faster.

(One) **set** use case: Use a set to make the elements in your list unique

```
In [2]: genes = ['Epas1', 'Cxcl14', 'Dcn', 'Epo', 'Epas1',  
                 'Dcn', 'Dcn', 'Gata3']  
print(set(genes))
```

```
{'Epo', 'Cxcl14', 'Gata3', 'Epas1', 'Dcn'}
```


(One) set use case: Use a set to make the elements in your list unique

```
In [2]: genes = ['Epas1', 'Cxcl14', 'Dcn', 'Epo', 'Epas1',  
                 'Dcn', 'Dcn', 'Gata3']  
print(set(genes))
```

```
{'Epo', 'Cxcl14', 'Gata3', 'Epas1', 'Dcn'}
```

This can be useful when you want to iterate over lists and want to avoid repeatedly iterating over the same element

Day 3, Exercise 1 (~30 min)

Find the number of unique genres in the file
250.imdb

#	Votes	Rating	Year	Runtime	URL	Genres	Title
126807		8.5	1957	5280	https://images-na.ssl-images...	Drama,War	Paths of Glory
71379		8.2	1925	4320	https://images-na.ssl-images...	Adventure,Comedy,Drama,Family	The Gold

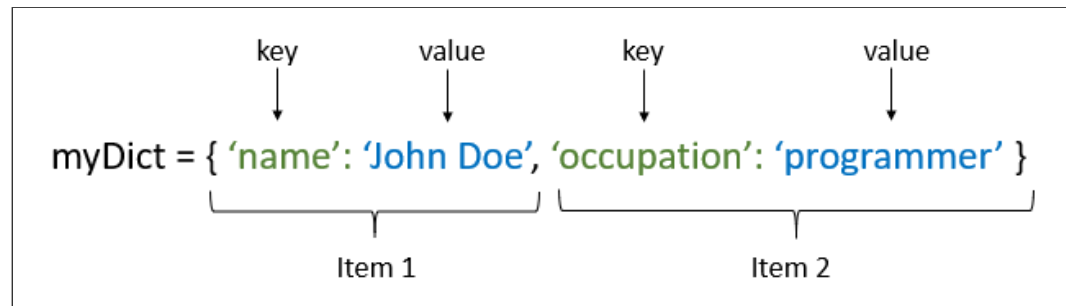
- Canvas -> Modules -> Day 3 -> IMDb exercise - day 3 -> 1
- Take a break after the exercise (~10 min)

Session 2

- Lecture: Data type `dictionary`
- Ex2: IMDb exercise - Find the number of movies per genre + Extra
- PyQuiz 3.1

New data type: **dictionary**

- A dictionary is an unordered, mutable collection of key-value pairs.
- Dictionaries are mutable
- Each key in a dictionary must be unique and immutable (the same as **set**), while the values associated with keys can be of any data type and can be duplicated



Syntax:

```
d = {} # Create an empty dictionary
```

```
d = {'key1':1, 'key2':2, 'key3':3} # create a populated dictionary
```

```
In [45]: myDict = {'drama': 4,  
                  'thriller': 2,  
                  'romance': 5}  
myDict
```

```
Out[45]: {'drama': 4, 'thriller': 2, 'romance': 5}
```

Basic operations on Dictionaries

Dictionary	
<code>len(d)</code>	Number of items
<code>d[key]</code>	Returns the item <i>value</i> for key <i>key</i>
<code>d[key] = value</code>	Updating the mapping for <i>key</i> with <i>value</i>
<code>del d[key]</code>	Delete key from d
<code>key in d</code>	Membership tests
<code>d.keys()</code>	Returns an iterator on the keys
<code>d.values()</code>	Returns an iterator on the values
<code>d.items()</code>	Returns an iterator on the pair (key, value)

```
In [3]: myDict = {'drama': 4,  
                 5: 2,  
                 'romance': 5}  
myDict
```

```
Out[3]: {'drama': 4, 5: 2, 'romance': 5}
```

```
In [6]: list(myDict.items())
```

```
Out[6]: [('drama', 4), (5, 2), ('romance', 5)]
```

Live Exercise

```
In [7]: myDict = {'drama': 182,  
                  'war': 30,  
                  'adventure': 55,  
                  'comedy': 46,  
                  'family': 24,  
                  'animation': 17,  
                  'biography': 25}
```

```
In [14]: myDict['comedy']
```

```
Out[14]: 47
```

- How many genres are in this dictionary?
- How many movies are in the **comedy** genre?
- You're not interested in biographies, delete this entry
- You're interested in fantasy; add that we have **29** movies in the **fantasy** genre to this dictionary.
- Which genres are listed in this dictionary after the change?
- You remembered another comedy movie; increase the number of movies in the **comedy** genre by one.

How to add a new key to a dictionary

```
In [51]: myDict['newkey'] = 25  
myDict
```

```
Out[51]: {'drama': 182,  
          'war': 30,  
          'adventure': 55,  
          'comedy': 47,  
          'family': 24,  
          'animation': 17,  
          'fantasy': 29,  
          'newkey': 25}
```


(One) dictionary use case - read a fasta file into a dictionary and perform operations on the sequences

(One) **dictionary** use case - read a fasta file into a dictionary and perform operations on the sequences

```
In [16]: fasta_dict = {'>seq1': 'ATGCATATATCGTC', '>seq2': 'GCGCGCGCGCTAGCT',  
                       '>seq3': 'ATATATATATATATATA'}  
  
for key, val in fasta_dict.items():  
    #Do some operation of your interest  
    print(key, val[1:5])
```

```
>seq1 TGCA  
>seq2 CGCG  
>seq3 TATA
```

IMDb Exercise 2: How to count occurrences of each genre

```
In [ ]: genreList = ["drama", "action", "drama", "horror", "thriller", "comedy",  
                    "drama", "comedy"]  
  
myDict = {}  
for g in genreList:  
    if not g in myDict:  
        myDict[g] = 1  
    else:  
        myDict[g] += 1  
  
myDict
```

Day 3, Exercise 2 (~50 min)

- Find the number of movies per genre
- (Extra) What is the average length of the movies (in hours and minutes) in each genre?
- Canvas -> Modules -> Day 3 -> IMDb exercise - day 3 -> 2&3

Take a break after the exercise (~10 min)

PyQuiz 3.1 - set, list and dictionary (before lunch)

Lunch

Session 3

- Lecture: Write our own functions
- Exercise 3: Functions

We have used many built-in functions

```
In [56]: print("Hello Python")
```

Hello Python

```
In [57]: len("ACCCCTTGAACCCC")
```

```
Out[57]: 14
```

```
In [58]: max([87, 131, 69, 112, 147, 55, 68, 130, 119, 50])
```

```
Out[58]: 147
```

We have used many built-in functions

```
In [56]: print("Hello Python")
```

Hello Python

```
In [57]: len("ACCCCTTGAACCCC")
```

```
Out[57]: 14
```

```
In [58]: max([87, 131, 69, 112, 147, 55, 68, 130, 119, 50])
```

```
Out[58]: 147
```

How to write our own functions?

Syntax of function

```
def function_name(arg1, arg2, ...):  
    # Block of code  
    return result
```

```
In [2]: def SayHi(name):  
        print("Hi", name)
```

```
SayHi('Anna')  
SayHi('Mike')
```

```
Hi Anna  
Hi Mike
```



```

In [17]: fh          = open('../downloads/250.imdb', 'r', encoding = 'utf-8')
genreDict = {}

#Read movie durations into dictionary
for line in fh:
    #Loop through all lines and extract variables of interest
    if not line.startswith('#'):
        cols      = line.strip().split('|')
        genre     = cols[5].strip()
        glist     = genre.split(',')
        runtime   = cols[3]          # length of movie in seconds
        for entry in glist:
            if not entry.lower() in genreDict:
                genreDict[entry.lower()] = [] # add a key with an empty list
            genreDict[entry.lower()].append(int(runtime)) # append runtime
fh.close()

# Calculate the average duration of movies in the genre 'drama'
genre = "drama"
average = sum(genreDict[genre])/len(genreDict[genre]) # calculate average
hours   = int(average/3600)                             # format seconds to hours
minutes = (average - (3600*hours))/60                   # format seconds to minutes
reformattedTime = str(hours)+'h'+str(round(minutes))+ 'min'
print('The average length for movies in genre '+ genre +\
      ' is '+ reformattedTime)

```

The average length for movies in genre drama is 2h14min

Now let's convert this code into a function

In [22]: *# Copy the previous code here*

The average length for movies in genre comedy is 1h53min

```
In [23]: for genre in ['drama', 'horror', 'comedy']:
         average = sum(genreDict[genre])/len(genreDict[genre]) # calculate a
         hours   = int(average/3600) # format
         minutes = (average - (3600*hours))/60 # format seconds
         reformattedTime = str(hours)+'h'+str(round(minutes))+ 'min'
         print('The average length for movies in genre '+ genre +\
               ' is ' + reformattedTime)
```

The average length for movies in genre drama is 2h14min
The average length for movies in genre horror is 1h59min
The average length for movies in genre comedy is 1h53min

```
In [23]: for genre in ['drama', 'horror', 'comedy']:
         average = sum(genreDict[genre])/len(genreDict[genre]) # calculate a
         hours   = int(average/3600)                          # format
         minutes = (average - (3600*hours))/60                # format seconds
         reformattedTime = str(hours)+'h'+str(round(minutes))+ 'min'
         print('The average length for movies in genre '+ genre +\
               ' is ' + reformattedTime)
```

```
The average length for movies in genre drama is 2h14min
The average length for movies in genre horror is 1h59min
The average length for movies in genre comedy is 1h53min
```

```
In [63]: for genre in ['drama', 'horror', 'comedy']:
         print('The average length for movies in genre '+ genre +\
               ' is ' + formatSec(genre, genreDict))
```

```
The average length for movies in genre drama is 2h14min
The average length for movies in genre horror is 1h59min
The average length for movies in genre comedy is 1h53min
```

Why use functions?

```
for genre in ['drama', 'horror', 'comedy']:
    print('The average length for movies in genre '+ genre +\
          ' is ' + formatSec(genre, genreDict))
```

- Clearly structured tasks in code
- Re-usability, you can use the same code at different places in your script without repeating it
- Better structure, cleaner and more readable code

Scope

- Local variables - Variables within functions
- Global variables - Variables outside of functions

Scope

- Local variables - Variables within functions
- Global variables - Variables outside of functions

```
In [64]: WEIGHT = 5  
def addWeight(value):  
    return value + WEIGHT  
print(addWeight(4))
```

9

Scope

- Local variables - Variables within functions
- Global variables - Variables outside of functions

```
In [64]: WEIGHT = 5
def addWeight(value):
    return value + WEIGHT
print(addWeight(4))
```

9

```
In [65]: WEIGHT = 5
def changeWeight():
    WEIGHT = 10
    print("WEIGHT inside the function is", WEIGHT)
    return None
changeWeight()
print("WEIGHT outside the function is", WEIGHT)
```

```
WEIGHT inside the function is 10
WEIGHT outside the function is 5
```


Use external libraries in Python

External libraries are collections of pre-written code, e.g objects and functions, that are not included in the Python standard libraries.

Use external libraries in Python

External libraries are collections of pre-written code, e.g objects and functions, that are not included in the Python standard libraries.

```
In [24]: #use the import statement to import a specific library  
import math  
math.sqrt(5)
```

```
Out[24]: 2.23606797749979
```

Use external libraries in Python

External libraries are collections of pre-written code, e.g objects and functions, that are not included in the Python standard libraries.

```
In [24]: #use the import statement to import a specific library  
import math  
math.sqrt(5)
```

```
Out[24]: 2.23606797749979
```

```
In [27]: sqrt(5)
```

```
Out[27]: 3.0
```

Use external libraries in Python

External libraries are collections of pre-written code, e.g objects and functions, that are not included in the Python standard libraries.

```
In [24]: #use the import statement to import a specific library  
import math  
math.sqrt(5)
```

```
Out[24]: 2.23606797749979
```

```
In [27]: sqrt(5)
```

```
Out[27]: 3.0
```

```
In [28]: from math import sqrt  
sqrt(5)
```

```
Out[28]: 2.23606797749979
```

Why use libraries

- Using complex functions from libraries saves time & prevents bugs
- Re-usability
- Better structure and cleaner code

How to define your own libraries

A simple library is just a .py file with some python functions

```
In [72]: #Define your functions
def formatSec(seconds):
    """Converts seconds to hour & minute format"""
    hours      = seconds/3600
    minutes    = (seconds - (3600*int(hours)))/60
    return str(int(hours))+ 'h' + str(round(minutes))+ 'min'

def toSec(days, hours, minutes, seconds):
    """Converts day, hour, minutes, seconds to seconds"""
    total = 0
    total += days*60*60*24
    total += hours*60*60
    total += minutes*60
    total += seconds

    return str(total)+ 's'
```

```
In [29]: from myFunctions import formatSec, toSec  
         print(formatSec(3900))
```

1h5min

```
In [31]: toSec(days=3, hours=1, minutes=0, seconds=1)
```

```
Out[31]: '262801s'
```

Summary

- A function is a block of organized, reusable code that is used to perform a single, related action
- Variables within a function are local variables
- Variables outside of functions are global variables
- Functions can be organized in separate files as libraries and be imported to the main code

Day 3, Exercise 3 (~30 min)

- Canvas -> Modules -> Exercise 3 - functions

Take a break after the exercise (~10 min)

Session 4

- Lecture: Pass arguments from command line using `sys.argv` and string formatting
- Ex4: IMDb exercise - functions and `sys.argv`
- PyQuiz 3.2

How to pass arguments to Python script from the command line?

Not just

```
python myscript.py
```

But also

```
python myscript.py arg1 arg2
```

sys.argv

- Avoid hardcoding the filename in the code
- Easier to re-use code for different input files
- Uses command-line arguments
- Input is list of strings:
 - Position 1 (index 0): the program name
 - Position 2 (index 1): the first argument
 - Position 3 (index 2): the second argument
 - etc

How to use it

```
import sys

program_name = sys.argv[0]
arg1 = sys.argv[1] # index error if the first argument is not provided
in the command
arg2 = sys.argv[2] # index error if the second argument is not
provided in the command
```

Code to copy a text file - myscript.py with hardcoded input and output

Code to copy a text file - myscript.py with hardcoded input and output

```
In [79]: input_file = "../downloads/250.imdb"
output_file = "newfile.txt"

#open input file
with open(input_file, "r") as fi:
    #open output file
    with open(output_file, "w") as fo:
        #Loop through all input lines and write them to output
        for line in fi:
            fo.write(line)
```

Code to copy a text file - myscript.py with hardcoded input and output

```
In [79]: input_file = "../downloads/250.imdb"
output_file = "newfile.txt"

#open input file
with open(input_file, "r") as fi:
    #open output file
    with open(output_file, "w") as fo:
        #Loop through all input lines and write them to output
        for line in fi:
            fo.write(line)
```

```
In [81]: !ls -lah newfile.txt
```

```
-rw-r--r--@ 1 stefanbmeyer  staff   46K Nov 20 16:49 newfile.tx
t
```


To make this script more flexible, use `sys.argv` to define input and output

```
# Code that can deal with command line arguments
import sys

input_file = sys.argv[1]
output_file = sys.argv[2]

with open(input_file, "r") as fi:
    with open(output_file, "w") as fo:
        for line in fi:
            fo.write(line)
```

To make this script more flexible, use `sys.argv` to define input and output

```
# Code that can deal with command line arguments
import sys

input_file = sys.argv[1]
output_file = sys.argv[2]

with open(input_file, "r") as fi:
    with open(output_file, "w") as fo:
        for line in fi:
            fo.write(line)
```

```
python myscript.py ../downloads/250.imdb newfile.txt
```

String formatting

Format text for printing or for writing to file.

What we have been doing so far:

String formatting

Format text for printing or for writing to file.

What we have been doing so far:

```
In [11]: chrom = "5"
pos = 1235651
ref = "C"
alt = "T"
geno = "1/1"
info = chrom + ":" + str(pos) + "_" + ref + "-" + alt + \
" has genotype: " + geno
print(info)
```

```
5:1235651_C-T has genotype: 1/1
```

Other (better) ways of formatting strings:

f-strings (since python 3.6)

```
In [12]: chrom = "5"
pos = 1235651
ref = "C"
alt = "T"
geno = "1/1"
info = f"{chrom}:{pos}_{ref}-{alt} has genotype: {geno}"
print(info)
```

5:1235651_C-T has genotype: 1/1

Other (better) ways of formatting strings:

f-strings (since python 3.6)

```
In [12]: chrom = "5"
pos = 1235651
ref = "C"
alt = "T"
geno = "1/1"
info = f"{chrom}:{pos}_{ref}-{alt} has genotype: {geno}"
print(info)
```

5:1235651_C-T has genotype: 1/1

```
In [14]: info = chrom + ":" + str(pos) + "_" + ref + "-" + alt + \
        " has genotype: " + geno
print(info)
```

5:1235651_C-T has genotype: 1/1

format method

```
In [15]: chrom = "5"
pos = 1235651
ref = "C"
alt = "T"
geno = "1/1"
info = "{}:{}_{}-{} has genotype: {}".format(chrom, pos, ref, alt, geno)
print(info)
```

5:1235651_C-T has genotype: 1/1

`f-strings" formatting is recommended

It works for other data types as well

```
In [83]: genes = ["TP53", "COX2"]  
         lengths = [355, 458]  
         print(f"Lengths of genes {genes} are {lengths}")
```

Lengths of genes ['TP53', 'COX2'] are [355, 458]

It works for other data types as well

```
In [83]: genes = ["TP53", "C0X2"]  
lengths = [355, 458]  
print(f"Lengths of genes {genes} are {lengths}")
```

Lengths of genes ['TP53', 'C0X2'] are [355, 458]

```
In [84]: gene = "C0X1"  
exp_level = 45.123253  
print(f"Expression level of gene {gene} is {exp_level}")
```

Expression level of gene C0X1 is 45.123253

It works for other data types as well

```
In [83]: genes = ["TP53", "C0X2"]  
lengths = [355, 458]  
print(f"Lengths of genes {genes} are {lengths}")
```

Lengths of genes ['TP53', 'C0X2'] are [355, 458]

```
In [84]: gene = "C0X1"  
exp_level = 45.123253  
print(f"Expression level of gene {gene} is {exp_level}")
```

Expression level of gene C0X1 is 45.123253

```
In [85]: print(f"Expression level of gene {gene} is {exp_level:.2e}")
```

Expression level of gene C0X1 is 4.51e+01

It works for functions and operations as well

```
In [86]: seq = "ATCGTAGCCCATAGC"  
print(f"The length of sequence {seq} is {len(seq)}")
```

The length of sequence ATCGTAGCCCATAGC is 15

```
In [ ]: text = "a string with many words "  
print(f"the text \"{text}\" is divided in to a list {text.split()}, "  
      f"and it has {len(text.split())} elements")
```

The ancient way (for Python 2, but still working)

```
In [87]: gene = "COX1"  
exp_level = 45.123253  
print("Expression level of gene %s is %f"%(gene, exp_level))
```

Expression level of gene COX1 is 45.123253

Summary

- Use `sys.argv` to deal with arguments passed to the python script from the command line
 - `sys.argv[0]` is the program name
 - `sys.argv[1]` is is the first argument and so on
- `f-strings` formatting is a convenient and recommended way to format the string
 - Extra reading about string formatting:
https://www.w3schools.com/python/python_string_formatting.asp

Day 3, Exercise 4 (~30 min)

- **Restructure and write the output to a new file**
- Canvas -> Modules -> Day 3 -> IMDb exercise - day 3 -> 4
- Work in pairs

PyQuiz 3.2

Project time

